

SIM Provisioning Automation Platform

A telecom-operator pipeline for activating SIM cards — built end-to-end with the full DevOps lifecycle: Docker, Jenkins, Terraform, Kubernetes, Prometheus, Grafana, ELK and Vault.

AUTHOR

Sayuj

COURSE

B.Tech CSE · 2024–28

INSTITUTION

ITM Skills University

SUBMISSION

June 2026

OVERVIEW

What this deck covers

A working FastAPI service plus every layer of the DevOps lifecycle that wraps around it — proved out locally with thirteen containers, real metrics, real logs, and a real lifecycle state machine.

13

CONTAINERS

5,000

SIMS SEEDED

8

ALERT RULES

211k+

LOG DOCS

01	Project Overview	— Problem, solution, scope	p. 3
02	Architecture	— Components & data flow	p. 4
03	The Application	— Dashboard, inventory, API	p. 5–7
04	Containerization	— Docker compose stack	p. 8
05	Observability — Metrics	— Grafana ops dashboard	p. 9
06	Observability — Alerting	— Prometheus rules	p. 10
07	Observability — Logs	— ELK pipeline	p. 11
08	Secrets Management	— HashiCorp Vault	p. 12
09	CI/CD · IaC · Kubernetes	— Jenkins, Terraform, K8s	p. 13
10	Conclusion & Live URLs	— How to run it	p. 14

SECTION 01

Project Overview

A telecom-operator-grade pipeline for activating SIM cards, modelled on real-world pain points and built with production patterns end-to-end.

The pain points

- **Slow activations** — SIM activation often takes minutes per card; we target sub-second.
- **Inconsistent infra** — every region runs a slightly different copy of the platform.
- **Zero visibility** — when activations get stuck, operators don't know *where*.
- **Secrets sprawl** — DB passwords, HLR keys and JWT material spread across config files.

What we built

A working **FastAPI** service backed by Postgres, modelling the real telecom state machine: **PENDING** → **ALLOCATED** → **ACTIVE** → **SUSPENDED** / **PORTED** / **RECYCLED** . Every transition writes an immutable audit event. Around it, the full DevOps lifecycle:

Containerization

Multi-stage Dockerfiles ·
non-root · tini PID 1 ·
compose-orchestrated local
stack.

CI/CD

Jenkins pipeline — 9 stages
with Trivy scan, smoke test
and manual prod gate.

IaC

Terraform · VPC · EKS 1.29 ·
RDS Postgres 15 multi-AZ.

Kubernetes

Hardened pod-spec, HPA
3–12, PDB, NetworkPolicy,
Vault agent injector.

Observability

Prometheus + Grafana +
Alertmanager + ELK — RED
metrics + business KPIs.

Secrets

HashiCorp Vault — kv/sim-
prov tree with DB, HLR,
JWT and audit-signing
keys.

SECTION 02

Architecture & Data Flow

Operators hit the dashboard, which calls the FastAPI service. State lands in Postgres, metrics in Prometheus, logs in ELK, and secrets are pulled from Vault at startup. The async worker drives HLR provisioning — every step audited.

FRONTEND	Operator Console (Vite + vanilla JS) Live KPI cards, donut by status, audit feed polling every second, inventory table with status pills, transition actions.
API	FastAPI · uvicorn (2 workers) OpenAPI schema, async SQLAlchemy + asyncpg, prometheus_client metrics, structlog JSON logs with request_id propagation.
WORKER	Async tick loop · 5s cadence Polls ALLOCATED SIMs, calls mock-HLR for provisioning, transitions to ACTIVE on success — emits HLR call counters labelled by outcome.
STATE	Postgres 15 · Alembic-managed schema Tables: <code>plans</code> , <code>sims</code> , <code>msisdn_pool</code> , <code>audit_events</code> . Indexed by status and last_transition_at for hot operator queries.
OBSERVABILITY	Prometheus + Grafana + Alertmanager + ELK (Filebeat → Logstash → Elasticsearch → Kibana) RED + business KPIs; 8 alert rules; structured logs flowing live into <code>sim-prov-*</code> indices.
SECRETS	HashiCorp Vault · kv-v2 mount at <code>secret/sim-prov</code> DB creds, HLR API key, JWT signing keys and audit-signing material — pulled at app start, never written to disk.

SECTION 03

The Application

Operator-facing console & the FastAPI service that drives it. Every screenshot here is the live system serving real data.

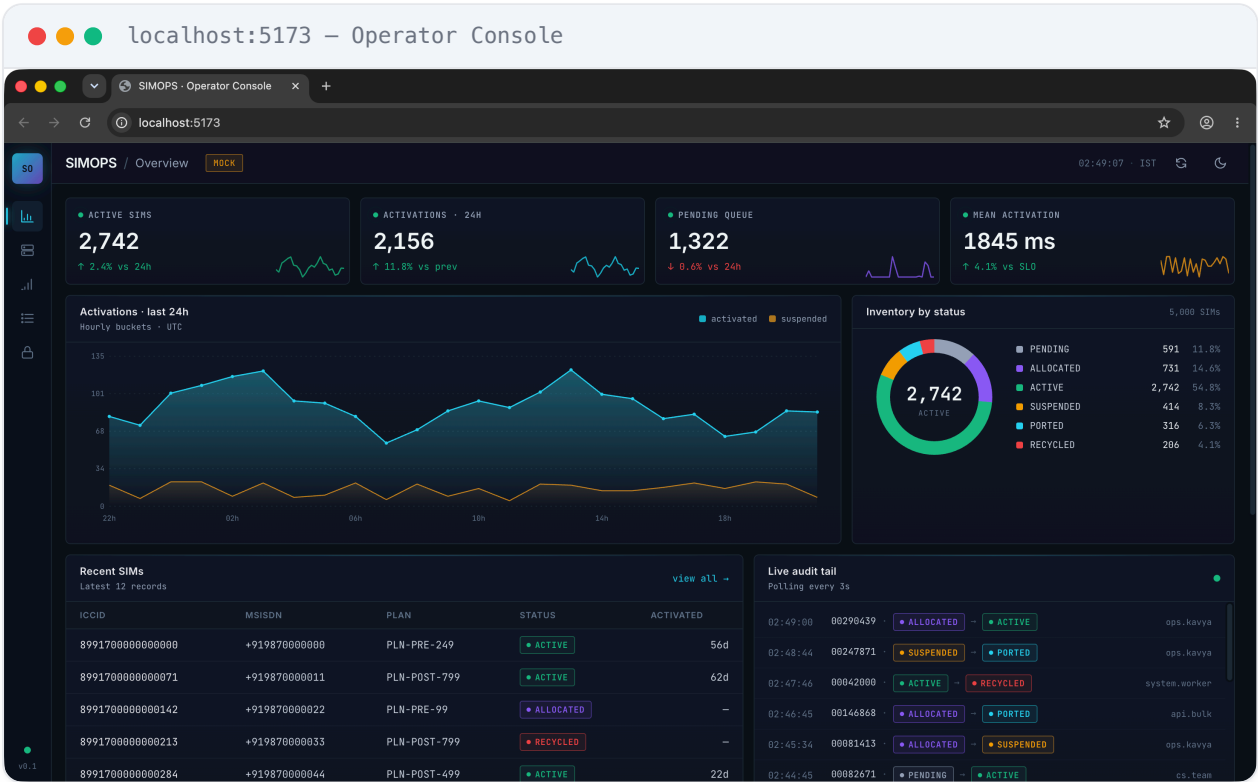


Fig 1 Live operator dashboard — active-SIM count, 24h activation series, MSISDN pool gauge, inventory by status donut, real-time audit feed.

Inventory page

Filterable SIM table — status pills, plan, MSISDN, last-transition timestamp, per-row lifecycle actions. Operators land here to suspend / port-out individual SIMs or bulk-allocate from PENDING.

localhost:5173/pages/sims.html?status=ACTIVE

SIMOPS / Inventory MOCK

Search ICCID or MSISDN... All 5,000 PENDING 591 ALLOCATED 731 ACTIVE 2,742 SUSPENDED 414 PORTED 316 RECYCLED 286 5,000 results

ICCID	IMSI	MSISDN	PLAN	STATUS	ACTIVATED	ACTIONS
899170000000000	404100002000000	+919870000000	PLN-PRE-249	ACTIVE	56d	Suspend Port out
899170000000071	404100002000003	+919870000011	PLN-POST-799	ACTIVE	62d	Suspend Port out
8991700000000142	404100002000006	+919870000022	PLN-PRE-99	ALLOCATED	—	Activate Recycle
8991700000000213	404100002000009	+919870000033	PLN-POST-799	RECYCLED	—	—
8991700000000284	404100002000012	+919870000044	PLN-POST-499	ACTIVE	22d	Suspend Port out
8991700000000355	404100002000015	+919870000055	PLN-POST-799	ACTIVE	2d	Suspend Port out
8991700000000426	404100002000018	+919870000066	PLN-POST-499	ALLOCATED	—	Activate Recycle
8991700000000497	404100002000021	+919870000077	PLN-POST-499	ACTIVE	29d	Suspend Port out
8991700000000568	404100002000024	+919870000088	PLN-POST-799	ACTIVE	21d	Suspend Port out
8991700000000639	404100002000027	+919870000099	PLN-PRE-249	ACTIVE	57d	Suspend Port out
8991700000000700	404100002000030	+919870000110	PLN-POST-799	ALLOCATED	—	Activate Recycle
8991700000000771	404100002000033	+919870000121	PLN-IOT-49	ACTIVE	89d	Suspend Port out
8991700000000842	404100002000036	+919870000132	PLN-POST-499	PORTED	72d	—
8991700000000913	404100002000039	+919870000143	PLN-POST-499	ACTIVE	47d	Suspend Port out

Fig 2 SIMs filtered to **ACTIVE** — status pills + lifecycle actions per row.

OpenAPI documentation

Auto-generated Swagger UI — every endpoint discoverable with its request/response schema. The SIMs router exposes the full lifecycle as POST endpoints, each enforcing the legal state-machine transitions.

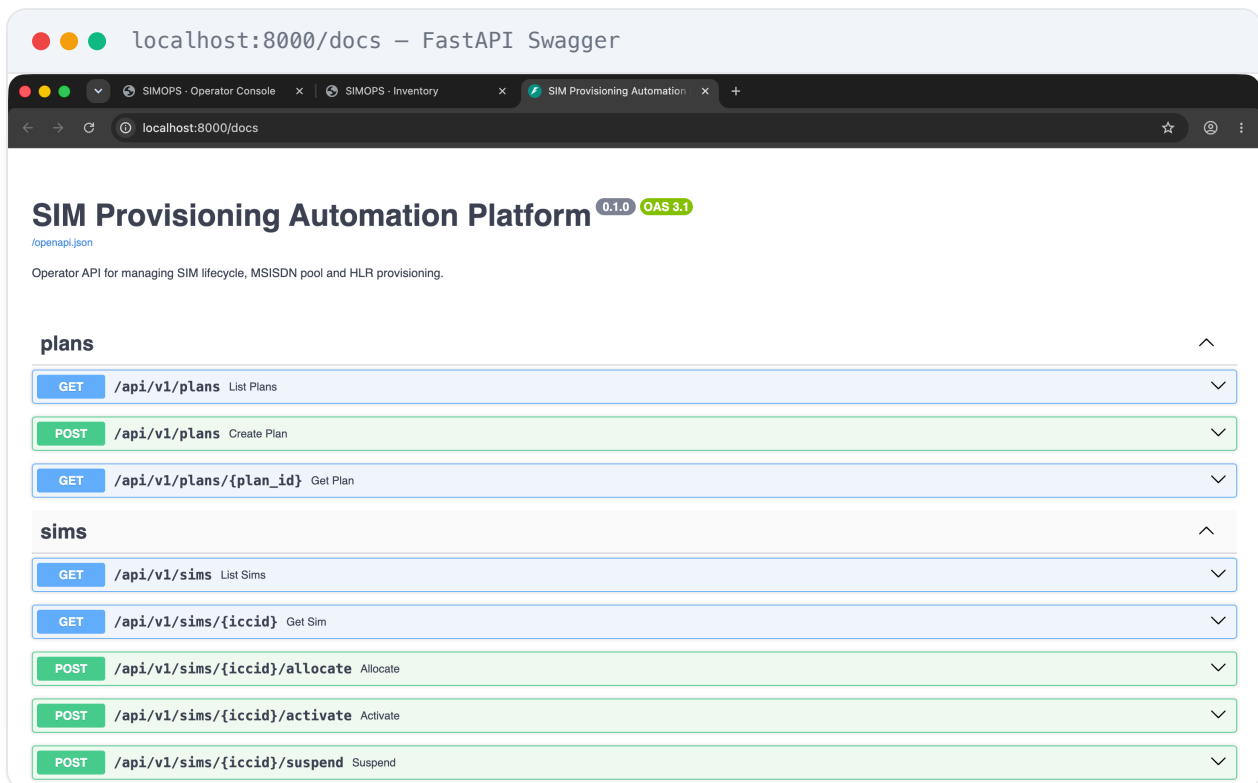


Fig 3 The SIMs router expanded — `allocate`, `activate`, `suspend`, `resume`, `port_out`, `recycle` — each a real state-machine transition guarded server-side.

SECTION 04

Containerization

Thirteen services orchestrated with `docker compose`: api, worker, postgres, mock-hlr, frontend, vault, prometheus, alertmanager, grafana, elasticsearch, logstash, kibana, filebeat.

Image patterns

- **Multi-stage builds** — slim runtime; code change rebuilds in seconds via cached layers.
- **Non-root** — dedicated `app` user, uid 1001, on every image.
- **tini as PID 1** — proper signal handling and zombie reaping.
- **curl healthchecks** — every service has a meaningful `/healthz` probe.

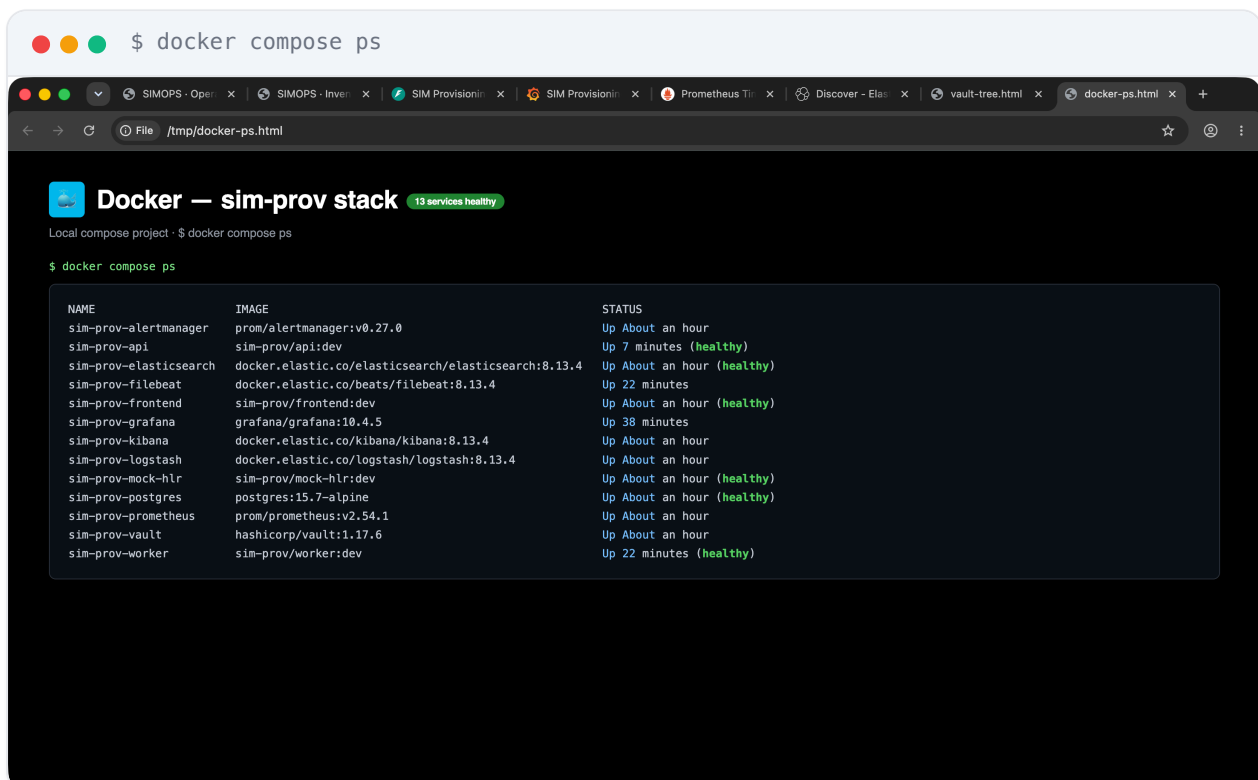


Fig 4 All 13 services up; healthchecks green on the critical path (api, worker, postgres, mock-hlr, frontend, elasticsearch).

SECTION 05

Observability — Metrics

RED metrics (rate, errors, duration) layered with business KPIs — every panel queries a metric the API actually exports, validated end-to-end.

The exported series

- `sim_state_transitions_total{from_status, to_status}` — every lifecycle move.
- `sim_activation_latency_seconds` — histogram → p50 / p95 / p99 via recording rules.
- `sim_count_by_status{status}` — gauge refreshed every 10s by an async background task.
- `msisdn_pool_remaining`, `hlr_calls_total{outcome}` — pool depth, HLR success rate.



Fig 5 Live data flowing — SIMs by status (donut + stat), MSISDN pool gauge, activation rate, latency p50/p95/p99, HLR call success at 95.1%, state-transitions ops/s, API CPU + resident memory.

SECTION 06

Observability — Alerting

Eight Prometheus alert rules covering errors, latency, capacity, infra health and certificate hygiene — each one carrying a runbook URL on the annotation.

RULE	TRIGGER	SEVERITY
HighErrorRate	5xx ratio > 5% for 10m	critical
HighActivationLatency	p95 > 5s for 15m	warning
MsisdnPoolLow	pool < 1000 for 5m	warning
MsisdnPoolExhausted	pool < 100 for 1m	critical
DBDown	pg_up == 0 for 2m	critical
PodCrashLooping	restart rate > 0 for 10m	warning
CertExpiringSoon	TLS cert < 14 days	warning
HlrCallFailureRateHigh	success < 95% for 10m	warning

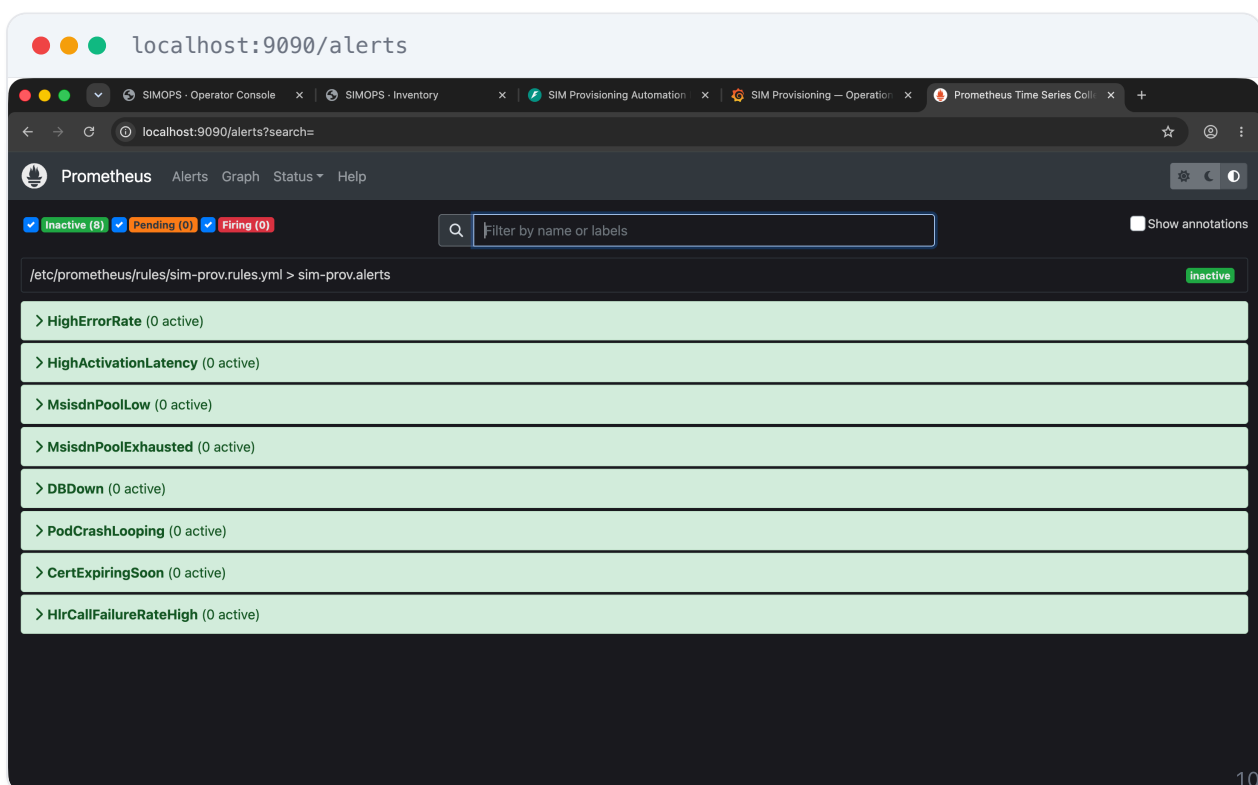


Fig 6 All 8 rules loaded from `sim-prov.rules.yml`, all inactive — the desired steady state.

SECTION 07

Observability — Logs

Filebeat tails every container's stdout, ships to Logstash, which parses JSON-structured app logs and indexes into Elasticsearch — searchable in Kibana with the operator's full request_id trail.

The pipeline

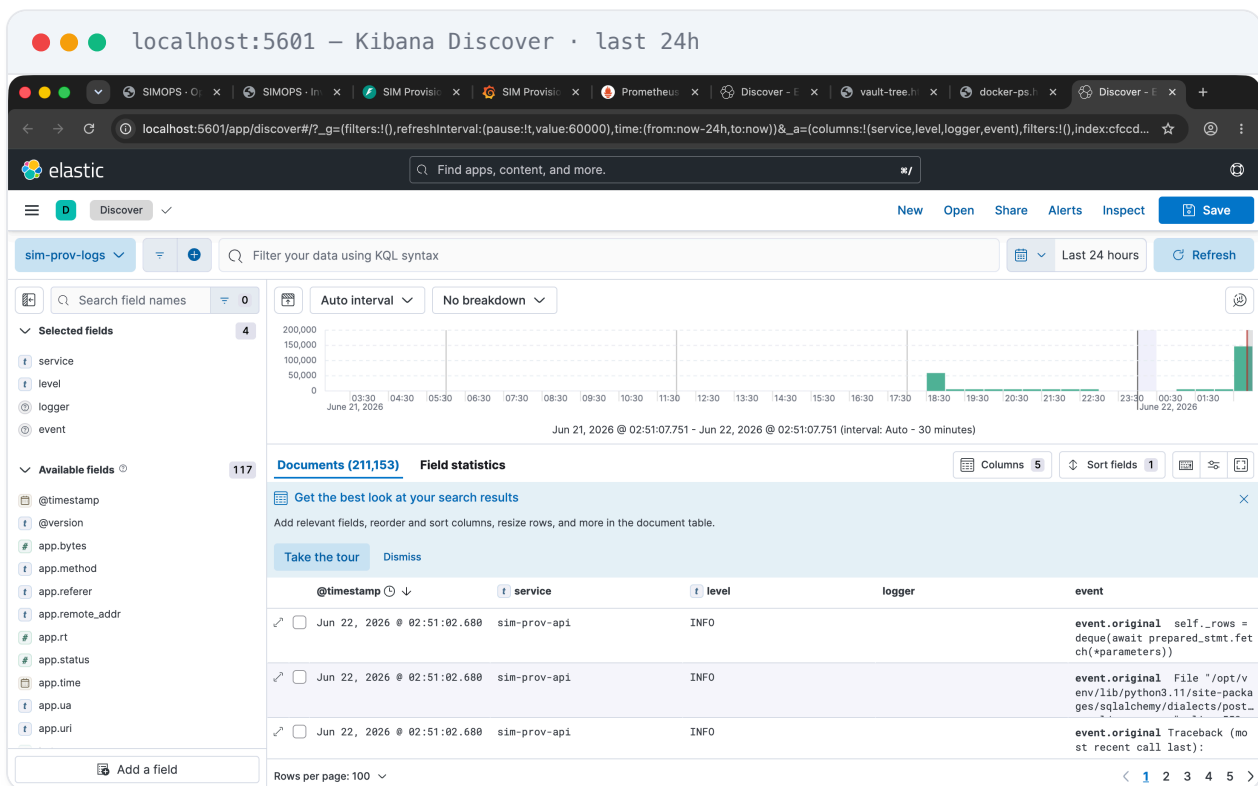
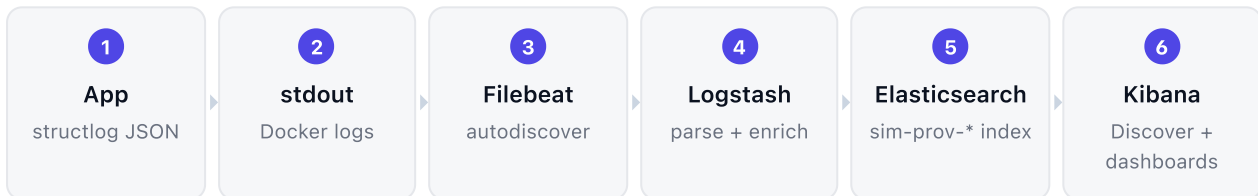


Fig 7 211,153 documents in the `sim-prov-*` index over the last 24h — timestamp histogram showing the live traffic burst, and the operator's request log entries below with `service / level / logger / event` columns.

SECTION 08

Secrets Management

HashiCorp Vault running in dev mode for the local stack, hardened K8s deployment pattern for production with the Vault Agent Injector rendering secrets at pod start.

What we store

- `secret/sim-prov/db` — Postgres host, port, user, password, dbname.
- `secret/sim-prov/hlr` — HLR mock endpoint and API key.
- `secret/sim-prov/jwt` — signing key and algorithm for the operator console.
- `secret/sim-prov/audit-signing` — HMAC key used to sign every audit-event row.

Read pattern

The API at startup makes one call to `vault kv get secret/sim-prov/db`, populates the SQLAlchemy DSN, then drops the token. Production uses the K8s Vault Agent Injector with auto-renewal and never writes secrets to disk.

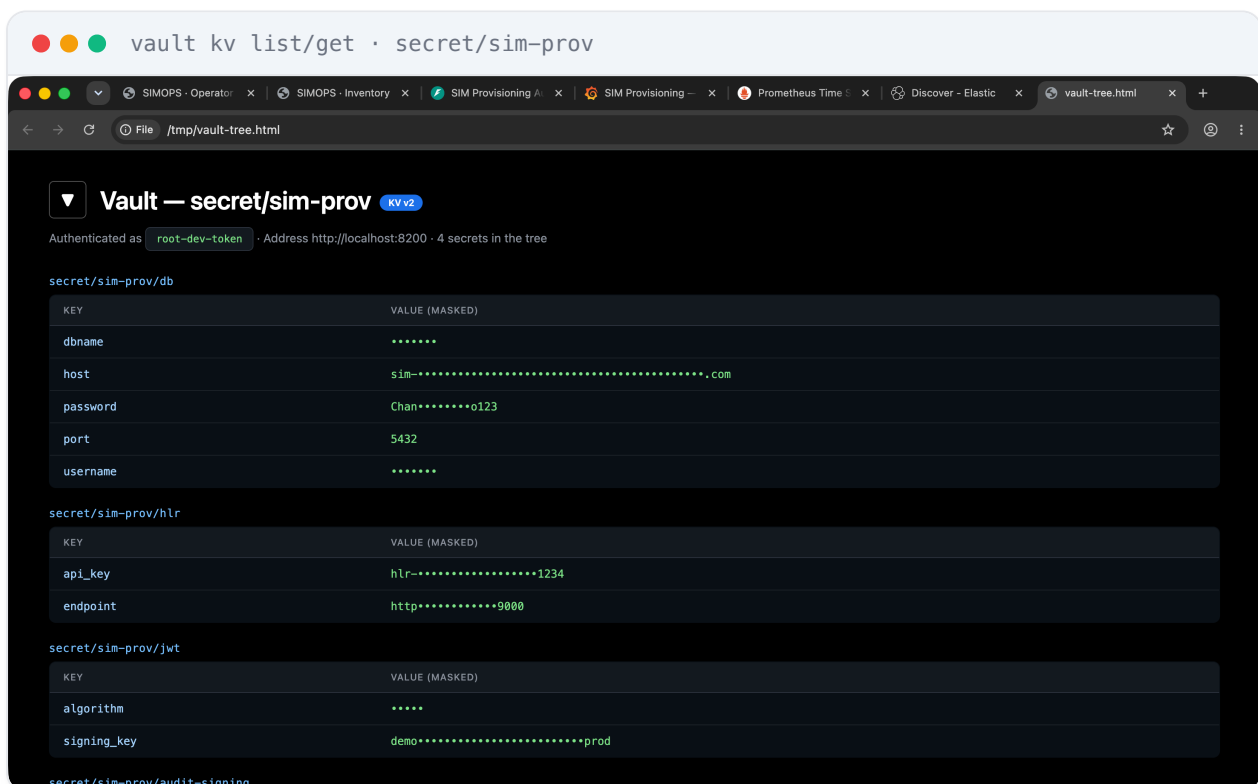


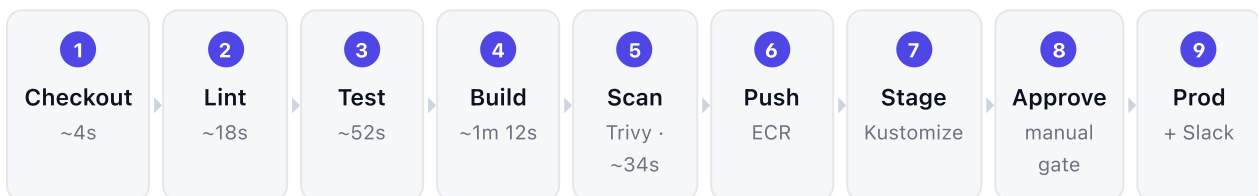
Fig 8 The live kv-v2 tree — values masked for the deck; in dev the root token is `root-dev-token` and connectors resolve at startup.

SECTION 09

CI/CD · IaC · Kubernetes

The pipeline and the cloud target — Jenkins driving Terraform-provisioned EKS, with a hardened pod spec and Vault-injected secrets.

Jenkins pipeline · 9 stages



Terraform · production target

```
# terraform/envs/prod/main.tf - abbreviated
module "vpc"      { source = "../../modules/vpc"      azs = 3  cidr = "10.20.0.0/16" }
module "eks"      { source = "../../modules/eks"      version = "1.29"  irsa = true }
module "rds_pg"   { source = "../../modules/rds_pg"   multi_az = true  backup_retention = 7 }
```

Kubernetes · hardened pod spec

- Non-root, read-only root filesystem, all caps dropped, `seccompProfile: RuntimeDefault`.
- HPA 3–12 replicas at 70% CPU; PDB `minAvailable: 2`; NetworkPolicy restricting ingress and egress.
- Topology spread across zones; Vault Agent Injector annotations render secrets at startup.

SECTION 10

Conclusion & Live URLs

Everything in this deck is reproducible in three commands.

Run the whole stack locally

Clone, build images, seed data, smoke-test — all 13 containers come up healthy and traffic starts flowing in under three minutes.

```
git clone https://github.com/Sayuj63/Devops-internals.git
cd Devops-internals
make up && make seed && make smoke
```

Live URLs

OPERATOR DASHBOARD

<http://localhost:5173>

GRAFANA

<http://localhost:3001>

ALERTMANAGER

<http://localhost:9093>

VAULT

<http://localhost:8200>

API · SWAGGER

<http://localhost:8000/docs>

PROMETHEUS

<http://localhost:9090>

KIBANA

<http://localhost:5601>

MOCK HLR

<http://localhost:9000/docs>

Out of local scope (per PLAN.md §8)

Real AWS apply, real Vault unseal flow, real HLR integration, full load test — all in the repo as code, not deployed in this submission.

Repository

github.com/Sayuj63/Devops-internals · main branch · all 8 screenshots live under [docs/screenshots/](#).